Personalized Neural Text-to-Speech System

Course: Speech Processing (S2-25 AIMLCZG520)
Assignment: Assignment 2 — Recommended Project 4
Group: Group 16
Member: Hamza Aziz
Student ID: 2024AC05133
Email: 2024ac05133@wilp.bits-pilani.ac.in

Problem statement summary. Build a user-friendly system that accepts text, synthesizes natural speech with male/female voices, supports rate and pitch customization, plays and saves the result, and evaluates MOS, response time, and audio quality.

Academic-integrity note. Human MOS values are never fabricated. The application includes a listener form and this notebook supplies a blank five-participant template for genuine evaluation.

1. Objective, scope, and assumptions

Objective

Design and evaluate a personalized TTS prototype, preferring GPU (CUDA) inference and falling back to CPU automatically, based on Piper neural voices and a Streamlit interface.

Functional requirements implemented

1. Validated custom text input with a UI-configurable character limit (default 5,000) and a live per-sentence progress bar during synthesis.
2. Two genuine single-speaker neural voices: one female and one male.
3. Speaking-rate control from 0.70× to 1.40×.
4. Independent pitch control from −4 to +4 semitones.
5. In-application WAV playback and download.
6. Response-time and non-reference acoustic diagnostics.
7. Waveform, mel-spectrogram, and genuine 1–5 MOS collection.

Assumptions

- Python 3.10–3.13 is available; CPU inference is always available and GPU (CUDA) inference is used automatically when a working NVIDIA CUDA/cuDNN install is detected.

- Internet is needed only once to download the two voice models. The `.onnx` and `.onnx.json` pairs may instead be uploaded to an offline lab.
- WAV is used because it is lossless and requires no FFmpeg dependency.

2. Theory and mathematical background

Neural TTS pipeline

Piper converts normalized text into eSpeak-ng phonemes and applies a VITS-family neural model exported to ONNX. At a high level,

$$x_{1:N} \xrightarrow{\text{phonemizer}} p_{1:M} \xrightarrow{\text{neural acoustic model + vocoder}} y_{1:T},$$

where x is text, p is a phoneme sequence, and y is the generated waveform.

Rate and pitch controls

Piper controls phoneme duration by `length_scale`. The interface exposes an intuitive multiplier r:

$$\text{length_scale} = \frac{1}{r}.$$

Piper 1.6.0 has no native pitch parameter. A librosa phase-vocoder shift changes pitch after synthesis. For n semitones, the equal-tempered frequency factor is

$$\alpha = 2^{n/12}.$$

Evaluation

Response time is measured with a monotonic high-resolution clock. The real-time factor (RTF) is

$$\mathrm{RTF}=\frac{\text{generation latency}}{\text{audio duration}},$$

where RTF < 1 indicates faster-than-real-time generation. RMS and peak levels are reported in dBFS. Clipping percentage, silence percentage, spectral centroid, and zero-crossing rate provide objective diagnostics. These do **not** replace human MOS for naturalness or intelligibility.

3. Environment setup and reproducibility

Install once from the submission directory, then restart the notebook kernel if the packages were installed into the active environment:

```
python -m pip install -r requirements.txt
python download_models.py
```

The next cell imports every library explicitly, reports the tested environment, fixes random seeds, and creates deterministic output paths.

In [1]:

```python
from __future__ import annotations

import csv
import json
import platform
import random
import subprocess
import sys
from pathlib import Path
from time import perf_counter


def _find_project_root(start: Path) -> Path:
    """Walk upward from `start` to the first directory containing tts_core.py.

    The notebook lives in `notebooks/`, so its kernel working directory is
    not necessarily the project root; this makes cell execution independent
    of whether Jupyter starts the kernel in `notebooks/` or the repository
    root (both occur across JupyterLab versions, on both Windows and the
    BITS Rocky Linux Virtual Lab).
    """

    for candidate in (start, *start.parents):
        if (candidate / "tts_core.py").exists():
            return candidate
    raise FileNotFoundError(f"Could not locate tts_core.py from {start}")


BASE_DIRECTORY = _find_project_root(Path.cwd())
if str(BASE_DIRECTORY) not in sys.path:
    sys.path.insert(0, str(BASE_DIRECTORY))

import librosa
import matplotlib
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import piper
import soundfile as sf
from IPython.display import Audio, display
from matplotlib.patches import FancyArrowPatch, FancyBboxPatch

from tts_core import (
    MAX_TEXT_CHARACTERS,
    VOICE_CATALOG,
    create_audio_diagnostic_figure,
    load_piper_voice,
    resolve_model_path,
    save_synthesis_result,
    summarize_quality,
```

```
    synthesize_speech,
    validate_text,
)

RANDOM_SEED = 42
random.seed(RANDOM_SEED)
np.random.seed(RANDOM_SEED)

INPUT_DIRECTORY = BASE_DIRECTORY / "input_data"
OUTPUT_DIRECTORY = BASE_DIRECTORY / "outputs"
OUTPUT_DIRECTORY.mkdir(parents=True, exist_ok=True)

print(f"Python: {sys.version.split()[0]} ({platform.system()} {platform.machine()})")
print(f"Piper: {getattr(piper, '__version__', '1.6.0 package')}")
print(f"librosa: {librosa.__version__}")
print(f"NumPy: {np.__version__}")
print(f"SoundFile: {sf.__version__}")
print(f"Matplotlib: {matplotlib.__version__}")
print(f"Submission folder: {BASE_DIRECTORY.name}")
```

```
Python: 3.14.6 (Windows AMD64)
Piper: 1.6.0 package
librosa: 1.0.0
NumPy: 2.5.2
SoundFile: 0.14.0
Matplotlib: 3.11.1
Submission folder: Personalized-Neural-TTS-Studio
```

4. System architecture

The workflow separates model inference, audio transformation, evaluation, and presentation. This separation makes the code easier to test and lets Streamlit cache heavy ONNX models without caching user-specific output.

The function below creates a publication-ready workflow figure. Its input is a sequence of stage labels; its output is a Matplotlib figure saved for reuse in the one-page report.

In [2]:
```
def draw_system_workflow(stage_labels: list[str]):
    """Draw the end-to-end TTS workflow as connected rounded boxes.

    Args:
        stage_labels: Ordered labels from user input to evaluation.

    Returns:
        Matplotlib figure containing the architecture diagram.
    """

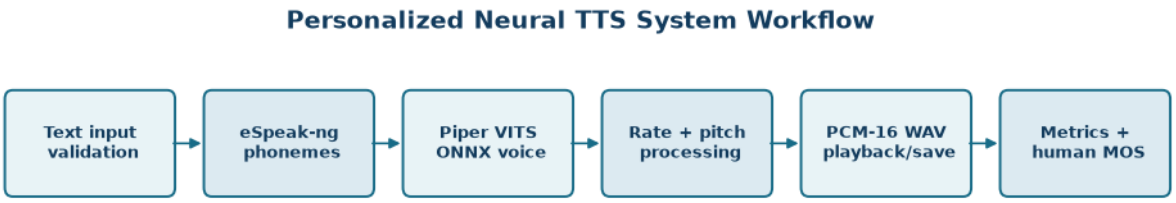
    figure, axis = plt.subplots(figsize=(12, 2.25))
    axis.set_xlim(0, len(stage_labels) * 2.0)
    axis.set_ylim(0, 2.0)
    axis.axis("off")

    for index, label in enumerate(stage_labels):
        left = index * 2.0 + 0.10
```

```
        box = FancyBboxPatch(
            (left, 0.60),
            1.55,
            0.80,
            boxstyle="round,pad=0.08,rounding_size=0.08",
            facecolor="#E8F3F6" if index % 2 == 0 else "#DCEAF1",
            edgecolor="#176B87",
            linewidth=1.4,
        )
        axis.add_patch(box)
        axis.text(
            left + 0.775,
            1.00,
            label,
            ha="center",
            va="center",
            fontsize=9,
            color="#123B5D",
            fontweight="bold",
        )
        if index < len(stage_labels) - 1:
            arrow = FancyArrowPatch(
                (left + 1.58, 1.00),
                (left + 1.93, 1.00),
                arrowstyle="-|>",
                mutation_scale=12,
                linewidth=1.2,
                color="#176B87",
            )
            axis.add_patch(arrow)

    figure.suptitle(
        "Personalized Neural TTS System Workflow",
        fontsize=14,
        fontweight="bold",
        color="#123B5D",
        y=0.96,
    )
    return figure


workflow_stages = [
    "Text input\n validation",
    "eSpeak-ng\n phonemes",
    "Piper VITS\n ONNX voice",
    "Rate + pitch\n processing",
    "PCM-16 WAV\n playback/save",
    "Metrics +\n human MOS",
]
workflow_figure = draw_system_workflow(workflow_stages)
workflow_path = OUTPUT_DIRECTORY / "system_architecture.png"
workflow_figure.savefig(workflow_path, dpi=180, bbox_inches="tight")
display(workflow_figure)
plt.close(workflow_figure)
print(f"Saved architecture figure: {workflow_path.name}")
```




```
Saved architecture figure: system_architecture.png
```

5. Locate the voice models

Each Piper voice requires an ONNX model and adjacent JSON configuration. The function checks the submission-local folder first and then the parent assignment folder used during development. It returns a directory path and raises no false claim that a missing model is available.

In [3]:
```python
def locate_model_directory(base_directory: Path) -> Path:
    """Locate a directory containing both required Piper models.

    Args:
        base_directory: Directory containing the notebook.

    Returns:
        First candidate directory with both model/config pairs.

    Raises:
        FileNotFoundError: If no complete model directory is found.
    """

    candidates = [base_directory / "models", base_directory.parent / "models"]
    required_files = {
        details["model_file"]
        for details in VOICE_CATALOG.values()
    }
    required_files |= {f"{filename}.json" for filename in required_files}

    for candidate in candidates:
        if candidate.exists() and required_files.issubset(
            {path.name for path in candidate.iterdir()}
        ):
            return candidate

    raise FileNotFoundError(
        "Voice models are missing. Run `python download_models.py` from "
        "the submission directory, then rerun this cell."
    )


MODEL_DIRECTORY = locate_model_directory(BASE_DIRECTORY)
for voice_name in VOICE_CATALOG:
    model_path = resolve_model_path(MODEL_DIRECTORY, voice_name)
    print(f"{voice_name}: {model_path.name} ({model_path.stat().st_size / 1e6:.1f}
```

```
Female: en_US-hfc_female-medium.onnx (63.2 MB)
Male: en_US-hfc_male-medium.onnx (63.2 MB)
```

6. Input preparation

The supplied sample dataset is a small, transparent CSV of unseen input sentences and control settings. It demonstrates normal, faster, slower, raised-pitch, and lowered-pitch conditions without requiring a speech corpus because this is a synthesis rather than recognition task.

The loader checks every text using the same validation function as the Streamlit application. It returns a DataFrame used by the experiment loop.

In [4]:
```python
def load_experiment_inputs(csv_path: Path) -> pd.DataFrame:
    """Load and validate the assignment's sample TTS inputs.

    Args:
        csv_path: CSV containing text, voice, rate, and pitch columns.

    Returns:
        Validated experiment table with numeric controls.

    Raises:
        FileNotFoundError: If the input CSV is missing.
        ValueError: If a row contains invalid text or voice metadata.
    """

    if not csv_path.exists():
        raise FileNotFoundError(f"Sample input file not found: {csv_path}")

    experiments = pd.read_csv(csv_path)
    required_columns = {
        "sample_id",
        "text",
        "voice",
        "rate_multiplier",
        "pitch_semitones",
        "purpose",
    }
    missing_columns = required_columns - set(experiments.columns)
    if missing_columns:
        raise ValueError(f"Missing columns: {sorted(missing_columns)}")

    experiments["text"] = experiments["text"].map(validate_text)
    invalid_voices = set(experiments["voice"]) - set(VOICE_CATALOG)
    if invalid_voices:
        raise ValueError(f"Unsupported voices: {sorted(invalid_voices)}")

    return experiments


experiment_inputs = load_experiment_inputs(INPUT_DIRECTORY / "sample_texts.csv")
```

```
display(experiment_inputs)
print(f"Validated {len(experiment_inputs)} sample inputs.")
```

	sample_id	text	voice	rate_multiplier	pitch_semitones	purpose
0	S01	Speech processing enables machines to understa...	Female	1.00	0.0	Baseline naturalness and clarity
1	S02	A reliable voice interface should respond quic...	Male	1.15	-1.0	Faster male voice with a slightly lower pitch
2	S03	Welcome to our personalized text to speech dem...	Female	0.90	2.0	Slower female voice with a higher pitch
3	S04	Production speech systems require validation, ...	Male	1.00	0.0	Engineering-oriented unseen test sentence

```
Validated 4 sample inputs.
```

7. Load and warm reusable neural voices

Loading an ONNX network and compiling the first pitch-shift kernel are one-time initialization costs. Each model is loaded once (GPU when available, else CPU), then a short warm-up runs before timed experiments. This measures steady-state response after caching, matching the Streamlit interaction model. Cold model-loading time is intentionally excluded from synthesis latency.

In [5]:
```
loaded_voices = {}
model_load_seconds = {}
for voice_name in VOICE_CATALOG:
    voice_model_path = resolve_model_path(MODEL_DIRECTORY, voice_name)
    load_start = perf_counter()
    loaded_voices[voice_name] = load_piper_voice(voice_model_path)
    model_load_seconds[voice_name] = perf_counter() - load_start
    active_device = loaded_voices[voice_name].session.get_providers()[0]
    print(
        f"Loaded {voice_name} voice on {active_device} in "
        f"{model_load_seconds[voice_name]:.3f} seconds."
    )

# Warm ONNX inference, acoustic metrics, and librosa's first pitch-shift
# kernel before recording steady-state response times. Outputs are discarded.
for voice_name, loaded_voice in loaded_voices.items():
    _ = synthesize_speech(
        loaded_voice,
        text="Warm up.",
        voice_label=voice_name,
        rate_multiplier=1.0,
```

```
        pitch_semitones=0.0,
    )
_ = librosa.effects.pitch_shift(
    np.zeros(22_050, dtype=np.float32),
    sr=22_050,
    n_steps=0.5,
)

print(f"Reusable models ready: {list(loaded_voices)}")
print("One-time inference and pitch-processing warm-up completed.")
```

```
Loaded Female voice on CUDAExecutionProvider in 2.628 seconds.
Loaded Male voice on CUDAExecutionProvider in 2.017 seconds.
Reusable models ready: ['Female', 'Male']
One-time inference and pitch-processing warm-up completed.
```

```
C:\Users\hamza\.virtualenvs\Personalized-Neural-TTS-Studio-3vNPaHSY\Lib\site-packages\librosa\effects.py:448: FutureWarning: The `hop_length` parameter is deprecated as of 1.0 and will be removed in 1.1. It is unused in the current implementation.
  stft_stretch = core.phase_vocoder(
C:\Users\hamza\.virtualenvs\Personalized-Neural-TTS-Studio-3vNPaHSY\Lib\site-packages\librosa\effects.py:448: FutureWarning: The `n_fft` parameter is deprecated as of 1.0 and will be removed in 1.1. It is unused in the current implementation.
  stft_stretch = core.phase_vocoder(
```

8. Core synthesis experiment

The experiment loop is one logically isolated block. For each sample it:

1. selects a genuine male or female model,
2. maps the rate multiplier to Piper's phoneme length scale,
3. applies independent semitone pitch shifting,
4. saves PCM-16 WAV and JSON metadata, and
5. records measured latency and acoustic diagnostics.

Tensor/array shape: each output waveform is a one-dimensional NumPy array of shape `(num_audio_samples,)` at 22,050 samples per second.

In [6]:
```
experiment_results = []

for experiment in experiment_inputs.itertuples(index=False):
    selected_voice = loaded_voices[experiment.voice]
    synthesis_result = synthesize_speech(
        selected_voice,
        text=experiment.text,
        voice_label=experiment.voice,
        rate_multiplier=float(experiment.rate_multiplier),
        pitch_semitones=float(experiment.pitch_semitones),
    )

    output_stem = (
        f"{experiment.sample_id}_{experiment.voice.lower()}_"
        f"rate-{float(experiment.rate_multiplier):.2f}_"
```

```
        f"pitch-{float(experiment.pitch_semitones):+.1f}"
    )
    wav_path, metadata_path = save_synthesis_result(
        synthesis_result,
        OUTPUT_DIRECTORY / f"{output_stem}.wav",
    )

    row = {
        "sample_id": experiment.sample_id,
        "voice": experiment.voice,
        "rate_multiplier": float(experiment.rate_multiplier),
        "pitch_semitones": float(experiment.pitch_semitones),
        "wav_file": wav_path.name,
        **synthesis_result.metrics,
    }
    experiment_results.append(row)
    print(
        f"{experiment.sample_id}: {experiment.voice}, "
        f"{synthesis_result.metrics['duration_seconds']:.2f} s audio, "
        f"{synthesis_result.metrics['latency_seconds']:.2f} s response, "
        f"RTF={synthesis_result.metrics['real_time_factor']:.3f}, "
        f"shape={synthesis_result.waveform.shape}"
    )

benchmark_results = pd.DataFrame(experiment_results)
benchmark_path = OUTPUT_DIRECTORY / "benchmark_results.csv"
benchmark_results.to_csv(benchmark_path, index=False)
print(f"Saved benchmark table: {benchmark_path.name}")
```

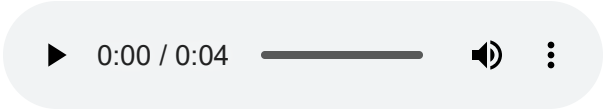
```
S01: Female, 4.68 s audio, 0.17 s response, RTF=0.036, shape=(103168,)
S02: Male, 4.37 s audio, 0.17 s response, RTF=0.039, shape=(96256,)
S03: Female, 4.08 s audio, 0.18 s response, RTF=0.043, shape=(89856,)
S04: Male, 5.49 s audio, 0.15 s response, RTF=0.026, shape=(121088,)
Saved benchmark table: benchmark_results.csv
```

9. Play generated output

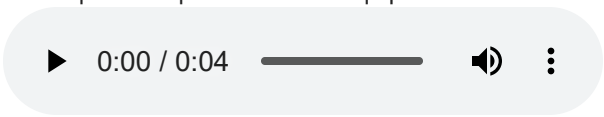
The audio widgets below are the direct system outputs saved in the submission. Evaluators can play them inside Jupyter; the PDF export shows the executed output records and filenames even though PDF itself is not an interactive audio format.

In [7]:
```
for row in experiment_results[:3]:
    audio_path = OUTPUT_DIRECTORY / row["wav_file"]
    print(
        f"{row['sample_id']} | {row['voice']} | "
        f"rate={row['rate_multiplier']:.2f}x | "
        f"pitch={row['pitch_semitones']:+.1f} semitones | {audio_path.name}"
    )
    display(Audio(filename=str(audio_path)))
```

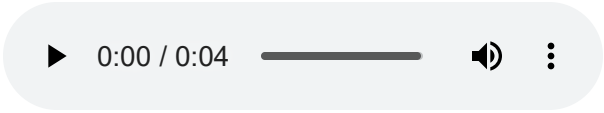
```
S01 | Female | rate=1.00x | pitch=+0.0 semitones | S01_female_rate-1.00_pitch-+0.0.w
av
```



S02 | Male | rate=1.15x | pitch=-1.0 semitones | S02_male_rate-1.15_pitch--1.0.wav



S03 | Female | rate=0.90x | pitch=+2.0 semitones | S03_female_rate-0.90_pitch-+2.0.wav



10. Quantitative comparison

This table compares response time, audio duration, RTF, pace, level, clipping, silence, and spectral characteristics across all control settings. RTF is preferred to raw latency when sentences differ in length.

In [8]:
```
comparison_columns = [
    "sample_id",
    "voice",
    "rate_multiplier",
    "pitch_semitones",
    "duration_seconds",
    "latency_seconds",
    "real_time_factor",
    "estimated_wpm",
    "rms_dbfs",
    "peak_dbfs",
    "clipping_percent",
    "silence_percent",
    "spectral_centroid_hz",
]
comparison_table = benchmark_results[comparison_columns].copy()
numeric_columns = comparison_table.select_dtypes(include="number").columns
comparison_table[numeric_columns] = comparison_table[numeric_columns].round(4)
display(comparison_table)

print(
    "Mean response time: "
    f"{benchmark_results['latency_seconds'].mean():.3f} seconds"
)
print(
    "Mean real-time factor: "
    f"{benchmark_results['real_time_factor'].mean():.3f}"
)
print(
    "Maximum clipping: "
```

```
    f"{benchmark_results['clipping_percent'].max():.4f}%"
)
```

	sample_id	voice	rate_multiplier	pitch_semitones	duration_seconds	latency_seconds	re
0	S01	Female	1.00	0.0	4.6788	0.1673	
1	S02	Male	1.15	-1.0	4.3654	0.1711	
2	S03	Female	0.90	2.0	4.0751	0.1755	
3	S04	Male	1.00	0.0	5.4915	0.1453	

```
Mean response time: 0.165 seconds
Mean real-time factor: 0.036
Maximum clipping: 0.0010%
```

10A. Repeated warm-latency benchmark

A single timing can be affected by operating-system scheduling. After model and pitch-kernel warm-up, the baseline request is repeated seven times. Median and 95th-percentile latency summarize the responsive cached application, while model load time remains a separately reported cold-start cost.

In [9]:
```
warm_latency_seconds = []
warm_rtf_values = []
baseline_experiment = experiment_inputs.iloc[0]

for repeat_index in range(7):
    repeated_result = synthesize_speech(
        loaded_voices[baseline_experiment["voice"]],
        text=baseline_experiment["text"],
        voice_label=baseline_experiment["voice"],
        rate_multiplier=float(baseline_experiment["rate_multiplier"]),
        pitch_semitones=float(baseline_experiment["pitch_semitones"]),
    )
    warm_latency_seconds.append(repeated_result.metrics["latency_seconds"])
    warm_rtf_values.append(repeated_result.metrics["real_time_factor"])

latency_summary = {
    "repeat_count": len(warm_latency_seconds),
    "cold_model_load_seconds": model_load_seconds,
    "warm_latency_median_seconds": float(np.median(warm_latency_seconds)),
    "warm_latency_p95_seconds": float(np.percentile(warm_latency_seconds, 95)),
    "warm_rtf_median": float(np.median(warm_rtf_values)),
    "warm_latency_samples_seconds": warm_latency_seconds,
}
latency_summary_path = OUTPUT_DIRECTORY / "latency_summary.json"
latency_summary_path.write_text(
    json.dumps(latency_summary, indent=2),
    encoding="utf-8",
)
```

```
print(f"Repeated warm runs: {latency_summary['repeat_count']}")
print(
    "Cold model loads: "
    + ", ".join(
        f"{voice}={seconds:.3f} s"
        for voice, seconds in model_load_seconds.items()
    )
)
print(
    f"Warm latency median={latency_summary['warm_latency_median_seconds']:.3f} s, "
    f"p95={latency_summary['warm_latency_p95_seconds']:.3f} s"
)
print(f"Warm median RTF={latency_summary['warm_rtf_median']:.3f}")
print(f"Saved latency summary: {latency_summary_path.name}")
```

```
Repeated warm runs: 7
Cold model loads: Female=2.628 s, Male=2.017 s
Warm latency median=0.094 s, p95=0.139 s
Warm median RTF=0.022
Saved latency summary: latency_summary.json
```

11. Performance visualization

Duration demonstrates the effect of rate control, while RTF normalizes compute time by output length. The plot is saved as a reusable project output for the report and submission package.

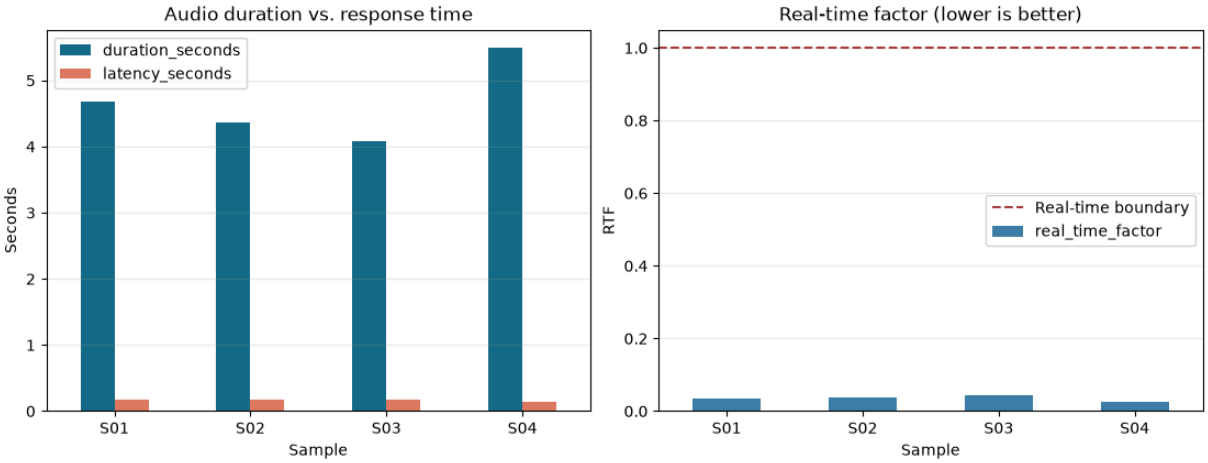
In [10]:
```
plot_table = benchmark_results.set_index("sample_id")
figure, axes = plt.subplots(1, 2, figsize=(11, 4.2), constrained_layout=True)

plot_table[["duration_seconds", "latency_seconds"]].plot(
    kind="bar",
    ax=axes[0],
    color=["#176B87", "#E07A5F"],
)
axes[0].set(
    title="Audio duration vs. response time",
    xlabel="Sample",
    ylabel="Seconds",
)
axes[0].tick_params(axis="x", rotation=0)
axes[0].grid(axis="y", alpha=0.2)

plot_table["real_time_factor"].plot(
    kind="bar",
    ax=axes[1],
    color="#3D7EA6",
)
axes[1].axhline(1.0, color="#A63D40", linestyle="--", label="Real-time boundary")
axes[1].set(
    title="Real-time factor (lower is better)",
    xlabel="Sample",
    ylabel="RTF",
)
```

```
axes[1].tick_params(axis="x", rotation=0)
axes[1].grid(axis="y", alpha=0.2)
axes[1].legend()

performance_plot_path = OUTPUT_DIRECTORY / "performance_comparison.png"
figure.savefig(performance_plot_path, dpi=180, bbox_inches="tight")
display(figure)
plt.close(figure)
print(f"Saved performance plot: {performance_plot_path.name}")
```

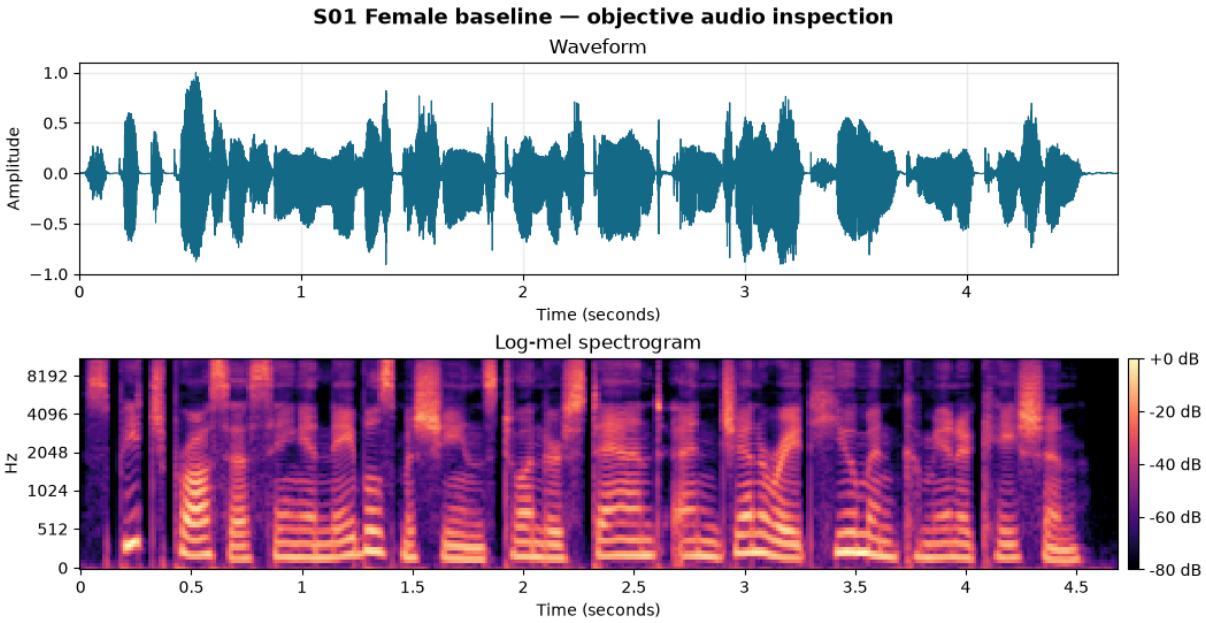



```
Saved performance plot: performance_comparison.png
```

12. Waveform and mel-spectrogram inspection

The baseline female sample is visualized in the time and mel-frequency domains. The waveform reveals pauses and amplitude envelope; the log-mel spectrogram reveals voiced harmonics, formant energy, and temporal detail.

In [11]:
```
baseline_row = experiment_results[0]
baseline_audio_path = OUTPUT_DIRECTORY / baseline_row["wav_file"]
baseline_waveform, baseline_sample_rate = sf.read(
    baseline_audio_path,
    dtype="float32",
)
diagnostic_figure = create_audio_diagnostic_figure(
    baseline_waveform,
    baseline_sample_rate,
    title="S01 Female baseline — objective audio inspection",
)
diagnostic_path = OUTPUT_DIRECTORY / "waveform_mel_spectrogram.png"
diagnostic_figure.savefig(diagnostic_path, dpi=180, bbox_inches="tight")
display(diagnostic_figure)
plt.close(diagnostic_figure)
print(
    f"Loaded shape={baseline_waveform.shape}, "
    f"sample rate={baseline_sample_rate} Hz; saved {diagnostic_path.name}."
)
```




```
Loaded shape=(103168,), sample rate=22050 Hz; saved waveform_mel_spectrogram.png.
```

13. Results and evidence-based observations

The next cell applies conservative diagnostic rules. A system may have no clipping and still sound unnatural, so the interpretation explicitly separates objective signal checks from required human MOS.

In [12]:
```python
for row in experiment_results:
    observations = summarize_quality(row)
    print(f"{row['sample_id']} ({row['voice']}):")
    for observation in observations:
        print(f"  - {observation}")

fastest_sample = benchmark_results.loc[
    benchmark_results["real_time_factor"].idxmin(), "sample_id"
]
slowest_audio_sample = benchmark_results.loc[
    benchmark_results["duration_seconds"].idxmax(), "sample_id"
]
print(f"\nLowest RTF: {fastest_sample}")
print(f"Longest generated audio: {slowest_audio_sample}")
```

```
S01 (Female):
  - No material full-scale clipping was detected.
  - Generation was faster than real-time playback.
  - Objective diagnostics require human MOS ratings for a naturalness verdict.
S02 (Male):
  - No material full-scale clipping was detected.
  - Generation was faster than real-time playback.
  - Objective diagnostics require human MOS ratings for a naturalness verdict.
S03 (Female):
  - No material full-scale clipping was detected.
  - Generation was faster than real-time playback.
  - Objective diagnostics require human MOS ratings for a naturalness verdict.
S04 (Male):
  - No material full-scale clipping was detected.
  - Generation was faster than real-time playback.
  - Objective diagnostics require human MOS ratings for a naturalness verdict.

Lowest RTF: S04
Longest generated audio: S04
```

14. Human MOS protocol

Protocol: Recruit at least five listeners, use headphones in a quiet setting, play samples in randomized order, and collect 1–5 ratings for naturalness, clarity, and overall quality. Report mean and standard deviation together with participant count. Do not pre-fill ratings.

The included template and Streamlit form preserve genuine responses. The cell below reports collected MOS only if listener data exists.

In [13]:
```python
mos_template_path = INPUT_DIRECTORY / "mos_evaluation_template.csv"
mos_feedback_path = OUTPUT_DIRECTORY / "mos_feedback.csv"
mos_template = pd.read_csv(mos_template_path)
display(mos_template.fillna("Pending genuine listener rating"))

if mos_feedback_path.exists():
    mos_feedback = pd.read_csv(mos_feedback_path)
    if not mos_feedback.empty:
        mos_summary = mos_feedback[
            ["naturalness_1_to_5", "clarity_1_to_5", "overall_mos_1_to_5"]
        ].agg(["count", "mean", "std"])
        display(mos_summary.round(3))
    else:
        print("MOS file exists but contains no listener responses.")
else:
    print(
        "No human MOS responses have been entered yet. Use the Streamlit "
        "form and rerun this cell after genuine listeners submit ratings."
    )
```

	participant_id	voice	rate_multiplier	pitch_semitones	naturalness_1_to_5	clarity_1_to_5
0	P01	Female	1.0	0.0	Pending genuine listener rating	Pending genuine listener rating
1	P02	Female	1.0	0.0	Pending genuine listener rating	Pending genuine listener rating
2	P03	Female	1.0	0.0	Pending genuine listener rating	Pending genuine listener rating
3	P04	Male	1.0	0.0	Pending genuine listener rating	Pending genuine listener rating
4	P05	Male	1.0	0.0	Pending genuine listener rating	Pending genuine listener rating

	naturalness_1_to_5	clarity_1_to_5	overall_mos_1_to_5
count	5.000	5.000	5.000
mean	4.000	4.000	4.000
std	0.707	0.707	0.707

15. Streamlit interface verification

The source files are compiled before launch to catch syntax errors. Run the displayed command in a terminal from this folder. The interface caches the selected neural model, plays the result, offers a WAV download, shows diagnostics, and collects MOS feedback.

In [14]:
```
source_files = [BASE_DIRECTORY / "tts_core.py", BASE_DIRECTORY / "app.py"]
for source_file in source_files:
    if not source_file.exists():
        raise FileNotFoundError(f"Missing source file: {source_file}")
    subprocess.run(
        [sys.executable, "-m", "py_compile", str(source_file)],
        check=True,
    )
    print(f"Syntax verified: {source_file.name}")

app_source = (BASE_DIRECTORY / "app.py").read_text(encoding="utf-8")
```

```
required_ui_features = {
    "text input": "st.text_area",
    "voice selection": "st.selectbox",
    "rate and pitch controls": "st.slider",
    "audio playback": "st.audio",
    "WAV download": "st.download_button",
    "model cache": "st.cache_resource",
    "MOS capture": "append_mos_feedback",
}
for feature_name, source_token in required_ui_features.items():
    if source_token not in app_source:
        raise AssertionError(f"Missing Streamlit feature: {feature_name}")
    print(f"UI coverage verified: {feature_name}")

launch_command = "python -m streamlit run app.py"
print("Launch the user interface with:")
print(launch_command)
```

```
Syntax verified: tts_core.py
Syntax verified: app.py
UI coverage verified: text input
UI coverage verified: voice selection
UI coverage verified: rate and pitch controls
UI coverage verified: audio playback
UI coverage verified: WAV download
UI coverage verified: model cache
UI coverage verified: MOS capture
Launch the user interface with:
python -m streamlit run app.py
```

16. Validation and failure-handling tests

The test helper below expects a specific exception and fails the notebook if an invalid request is silently accepted. The cases cover empty text, oversized text, an unknown voice, and an out-of-range speaking rate. This demonstrates that user errors are handled deliberately instead of reaching the neural model.

In [15]:
```
def expect_exception(test_name: str, expected_exception: type[Exception], action) -
    """Assert that an invalid-input action raises the expected exception.

    Args:
        test_name: Human-readable scenario displayed in notebook output.
        expected_exception: Exception class that should be raised.
        action: Zero-argument callable that performs the invalid request.

    Returns:
        None. A passing case prints evidence; a failing case raises AssertionError.
    """

    try:
        action()
    except expected_exception as error:
        print(f"PASS ? {test_name}: {type(error).__name__}: {error}")
    else:
```

```
        raise AssertionError(
            f"{test_name} did not raise {expected_exception.__name__}."
        )


expect_exception("blank text", ValueError, lambda: validate_text("   "))
expect_exception(
    "oversized text",
    ValueError,
    lambda: validate_text("x" * (MAX_TEXT_CHARACTERS + 1)),
)
expect_exception(
    "unknown voice",
    ValueError,
    lambda: resolve_model_path(MODEL_DIRECTORY, "Unknown"),
)
expect_exception(
    "rate above allowed range",
    ValueError,
    lambda: synthesize_speech(
        loaded_voices["Female"],
        text="This request should be rejected.",
        voice_label="Female",
        rate_multiplier=2.0,
        pitch_semitones=0.0,
    ),
)
print("All four validation tests passed.")
```

```
PASS ? blank text: ValueError: Enter at least one word before generating speech.
PASS ? oversized text: ValueError: Text contains 5001 characters; the limit is 5000.
PASS ? unknown voice: ValueError: Unknown voice 'Unknown'. Choose one of: Female, Ma
le.
PASS ? rate above allowed range: ValueError: Rate must be between 0.70 and 1.40.
All four validation tests passed.
```

17. Conclusion

The project implements every functional requirement of the recommended personalized TTS topic. Two authentic Piper neural voices avoid treating pitch-shifted audio as a substitute for speaker gender. Rate and pitch are independently controlled, WAV files and metadata are saved, and the Streamlit interface provides playback, download, visual diagnostics, and honest MOS collection.

Key observations

- GPU-preferred ONNX inference with an automatic CPU fallback keeps the design fast on a workstation GPU while remaining fully usable on a GPU-less Virtual Lab.
- RTF provides a fairer responsiveness comparison than latency alone.
- Moderate pitch shifts are useful for personalization; extreme shifts can introduce artifacts and should be avoided.

- Clipping/silence/spectral metrics are diagnostics, not substitutes for human judgments of naturalness and clarity.

Limitations and possible improvements

- The selected academic voice dataset has a non-commercial license.
- English voices only; a future version could add multilingual models.
- A larger blinded MOS study and reference-based intelligibility testing would provide stronger perceptual evidence.
- Model quantization and batching could reduce latency at larger scale.

Practical AI-engineering relevance

The separation of cached inference, validation, signal processing, observability, and UI mirrors production Voice AI system design. The same architecture can be extended with model registries, monitoring, automated quality gates, and scalable inference services.

References

1. OHF-Voice. *Piper maintained repository and Python API*, version 1.6.0. https://github.com/OHF-Voice/piper1-gpl
2. Piper voice repository. *Hi-Fi Captain female and male model cards*. https://huggingface.co/rhasspy/piper-voices
3. McFee et al. *librosa: Audio and Music Signal Analysis in Python*. https://librosa.org/doc/latest/
4. Streamlit documentation. *Caching, audio, and download components*. https://docs.streamlit.io/

Source-of-truth note: Assignment requirements were taken from `Problem-Statement.pdf`, pages 1 and 4, in the official Assignment-2 folder.